



Programação Estruturada em C
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
RELATÓRIO DE AULAS PRÁTICAS

Nome:Vitória Freitas

RA:2523279

Polo de matrícula:Vila Dirce-Carapicuíba

Local da realização da Aula Prática:Alphaville-Santana de Parnaíba

Ano da postagem:2025

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Programação Estruturada em C	Relatório 1
--	--	--------------------

Estrutura de um programa em C

Um programa em C vai ser escrito, utilizando uma ou mais funções. A função principal vai se chamar “main”, cada um dessa função irá retornar algo ou irá retornar 0.

Existem regras para serem seguidas em c, algumas delas seriam: todo programa em C, tem a função main, toda função precisa ter dois parênteses após o nome, as linhas de código no fim recebem um ponto e vírgula, códigos são executados da maneira que foram escritos.

A indentação em C tem sua importância, pois ela gera uma maior legibilidade para o código, mas caso não indente corretamente, o programa não gerará erro por conta disso. Por tanto, indentar em C não é algo obrigatório, mas sim uma boa prática.

Tipos de dados básicos em C

Tipos de dados básicos em C: int(inteiro), char(caracter), double(real de precisão simples), void(vazio).

Comandos básicos em C (printf() e scanf())

Printf() serve para mostrar mensagens na tela, scanf() para ler valores e guardar seu resultado na memória. Para mostrar um valor na tela precisa identificar de que tipo é esse valor, por exemplo se for inteiro precisa do “%d”, float “%f”, char “%c” de um único caractere. Para o programa ler um também precisa identificar o seu tipo, exemplo “&d” para inteiros, “&f” para float, “&c” para char.

Identificadores e variáveis.

Variável tem a função de guardar um valor dentro da memória, todas elas têm identificadores e um tipo.

Identificadores são os nomes que você dá para suas variáveis, eles devem ser diferentes, não pode ter duas variáveis com o mesmo nome. A linguagem C é “case-sensitive”, ou seja, tem diferença uma variável com letra maiúscula e uma com letra minúscula, é obrigatório que eles sejam diferentes dos comandos de C e só pode ser

inicializado com uma letra ou um “_”.

Atividade 1 do roteiro.

```
// Aula 1
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // variáveis
    char name[50];
    char email[50];
    int age;

    // leitura de dados
    printf("Digite seu nome: ");
    scanf("%s", &name);

    printf("Digite sua idade: ");
    scanf("%d", &age);

    printf("Digite seu e-mail: ");
    scanf("%s", &email);

    // mostrando os dados
    printf("Ola, %s! Voce tem %d anos, seu e-mail: %s\n", name, age, email);

    return 0;
}
```

Outras atividades feitas.

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // primeira atividade

    printf("Ola, mundo!\n");
}
```

```
// segunda atividade
```

```
float valor1, valor2, valor3, total;  
valor1 = 12.6;  
valor2 = 3.4;  
valor3 = 6.2;  
total = valor1 + valor2 + valor3;  
printf("Total: %.2f \n", total);
```

```
// terceira atividade
```

```
int idade;  
float altura;  
char inicial;
```

```
printf("Digite sua idade: ");  
scanf("%d", &idade);
```

```
printf("Digite sua altura: ");  
scanf("%f", &altura);
```

```
printf("Digite a inicial do seu nome: ");  
scanf(" %c", &inicial);
```

```
printf("\nIdade: %d, Altura: %.2f, Inicial: %c\n ", idade, altura, inicial);
```

```
// variáveis
```

```
char name[50];  
char email[50];  
int age;
```

```
// leitura de dados
```

```
printf("Digite seu nome: ");  
scanf("%s", &name);
```

```
printf("Digite sua idade: ");  
scanf("%d", &age);
```

```
printf("Digite seu e-mail: ");  
scanf("%s", &email);
```

```
// mostrando os dados
```

```
printf("Ola, %s! Voce tem %d anos, seu e-mail: %s\n", name, age, email);
```

```
return 0;
```

```
}
```

Referências

<https://lugarcerto.wordpress.com/2013/03/29/estrutura-basica-de-um-programa-em-c/>

<https://linguagemc.com.br/tipos-de-dados-em-c/>

<https://linguagemc.com.br/variaveis-em-linguagem-c/>

<https://learn.microsoft.com/pt-br/cpp/c-language/c-identifiers?view=msvc-170>

<https://www.inf.pucrs.br/~pinho/Laprol/IntroC/IntroC.htm>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Operadores Básicos em C	Relatório 2
--	---	--------------------

Operadores de atribuição

Operadores de atribuição atribuem o valor do operando à direita ao operando da esquerda, que é quem recebe o valor.

Exemplos: = *= /= %= += -= <=> &= ^= !=

Esses operadores de atribuição em C, podem atribuir valores ou transformá-los.

Operadores aritméticos em C

- + (soma), exemplo: A + B
- - (subtração), exemplo: A - B
- * (multiplicação), exemplo: A * B
- / (divisão inteira), exemplo: B / A
- % (módulo), exemplo: B % A

Operadores relacionais e lógicos

Relacionais:

- == (igual), exemplo: A == B
- != (diferente), exemplo: A != B
- > (maior do que), exemplo: A > B
- < (menor do que), exemplo: A < B
- >= maior ou igual, exemplo: A >= B
- <= menor ou igual, exemplo: A <= B

Lógicos:

- && (AND lógico), exemplo: A && B
- || (OR lógico), exemplo: A || B
- ! (NOT lógico), exemplo: !A

Operadores incremento e decremento

- ++ (incremento), exemplo: A++
- -- (decremento), exemplo: A--

Os operadores de incremento e decremento tem os pré e os pós-fixados, no pré (++a)

o incremento acontece antes que o valor seja usado, no pós (a++) acontece depois que o valor for utilizado.

Códigos-fonte do roteiro

```
// aula 2
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "Portuguese");

    // primeira atividade

    int a = 10, b = 3;

    printf("Soma: %d\n", a + b);
    printf("Subtracao: %d\n", a - b);
    printf("Multiplicacao: %d\n", a * b);
    printf("Divisao: %d\n", a / b);
    printf("Resto da divisao: %d\n", a % b);

    // segunda atividade

    int x = 5;

    x += 3;
    printf("Resultado de x += 3: %d\n", x);

    x *= 2;
    printf("Resultado de x *= 2: %d\n", x);

    // terceira atividade

    int idade = 18;

    printf("Idade igual a 18? %d\n", idade == 18);
    printf("Idade maior que 17 %d\n", idade > 17);
    printf("Idade diferente de 20? %d\n", idade != 20);

    // quarta atividade
    int A = 5, B = 10;
```

```
printf("a > 0 E b > 0: %d\n", (A > 0) && (B > 0));  
printf("a > 0 OU b < 0: %d\n", (A > 0) || (B < 0));  
printf("Nao (a > b): %d\n", !(A > B));
```

```
// quinta atividade
```

```
// variáveis
```

```
char name[50];
```

```
int age;
```

```
float n1, n2, media;
```

```
// entrada de dados
```

```
printf("Digite seu nome: ");
```

```
scanf(" %s", &name);
```

```
printf("Digite sua idade: ");
```

```
scanf("%d", &age);
```

```
printf("Digite duas notas: ");
```

```
scanf("%f %f", &n1, &n2);
```

```
media = (n1 + n2) / 2;
```

```
// resultado
```

```
printf("\nOla, %s!\n", name);
```

```
printf("Media: %.2f\n", media);
```

```
printf("Media >= 7? %d\n", media >= 7);
```

```
printf("Idade < 18? %d\n", age < 18);
```

```
return 0;
```

```
}
```

Referências

<https://learn.microsoft.com/pt-br/cpp/c-language/c-assignment-operators?view=msvc-170>

https://www.inf.ufpr.br/hexsel/ci067/02_operad.html

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Operações e Controle de Fluxo	Relatório 3
--	---	--------------------

Estruturas condicionais if

Estruturas condicionais são o que dão a capacidade ao programa de alterar o modo que o código irá se comportar dependendo das condições. E é para isso que utilizamos o if, ele irá tomar uma decisão com base em uma condição ou expressão, se a condição for verdadeira irá executar aquele bloco de código que está dentro dele, se for falsa aquele bloco vai ser ignorado.

Estruturas condicionais if-else

If-else funciona de maneira similar, porém se a condição do if for falsa, ele irá executar a que está dentro do else, caso seja verdadeira executa o que está dentro do if. Dessa maneira conseguimos construir programas mais robustos e complexos, que podem tomar decisões de diferentes em cenários diferentes.

Códigos-fonte do roteiro

Atividade proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // Variáveis

    char name[50];
    int age;
    float n1;
    float n2;
    float n3;
```

```

float average;

// Pegando dados

printf("Coloque seu nome: ");
scanf("%s", &name);

printf("Coloque sua idade: ");
scanf("%d", &age);

printf("Coloque sua primeira nota: ");
scanf("%f", &n1);

printf("Coloque sua segunda nota: ");
scanf("%f", &n2);

printf("Coloque sua terceira nota: ");
scanf("%f", &n2);

average = (n1 + n2 + n3) / 3;

// dependendo da média a mensagem executada será diferente

if (average >= 9){
    printf("Aprovado com excelente desempenho!");
} else if (average >= 7){
    printf("Aprovado, mas ainda pode melhorar.");
} else {
    printf("Reprovado. Continue se esforçando!");
}

return 0;
}

```

Exemplo 1:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

```

```

int main()
{
    // Atividade 1
    setlocale(LC_ALL, "Portuguese");

    int idade;
    printf("Digite sua idade: ");
    scanf("%d", &idade);
    // Mensagem caso idade seja maior ou igual a 18
    if(idade >= 18){
        printf("Voce é maior de idade.\n");
    }

    return 0;
}

```

Exemplo 2:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Atividade 2

    int numero;
    printf("Digite um número: ");
    scanf("%d", &numero);
    // Verificação se o número é par ou impar
    if(numero % 2 == 0){
        printf("Par");
    } else {
        printf("Impar");
    }

    return 0;
}

```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
    // Atividade 3

    int senhaDigitada;
    int senhaCorreta = 1234;

    printf("\nDigite a senha: ");
    scanf("%d", &senhaDigitada);
    // Caso senha diferente, acesso negado
    if(senhaDigitada == senhaCorreta){
        printf("Senha permitida.\n");
    } else {
        printf("Senha incorreta! tente novamente.\n");
    }

    return 0;
}
```

Exemplo 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

int main()
{
```

```
setlocale(LC_ALL, "Portuguese");

// Variável do tipo int
int nota;

printf("Digite a nota (0 a 10): ");
scanf("%d", &nota);
// nota maior ou igual a 7 está aprovado, entre 5 e 6 recuperação, menor que 5
reprovado
if (nota >= 7){
    printf("Aprovado.\n");
} else if (nota >= 5){
    printf("Em recuperação.\n");
} else {
    printf("Reprovado.\n");
}

return 0;
}
```

Referências

<https://www.freecodecamp.org/portuguese/news/instrucoes-if-else-em-c-explicadas/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Estruturas de Repetição em C (Laços)	Relatório 4
--	--	--------------------

Laços de repetição: while, do-while e for

While

Irá executar um bloco de código enquanto a condição for verdadeira, se for falsa desde o início, o laço será ignorado. A variável que vai ser testada dentro da condição, chamamos de variável de controle, quando ela se tornar falsa irá pular para fora do laço.

do-while

Com o do-while diferente do while, o bloco de código será executado pelo menos uma vez, mesmo que a condição seja falsa desde o início, isso acontece, pois a condição é testada apenas no final.

for

Muito utilizada quando se sabe quantas vezes irá se repetir o laço, esse laço utiliza uma variável para controlar a contagem do loop e também seu incremento. Essa estrutura faz a inicialização, incremento e encerramento do laço.

Break e Continue

O comando break ele cumpri a função de encerrar um laço completamente, o comando Continue ele não encerra o loop e sim pula para a próxima iteração, ignorando os comandos intermediários.

Switch-case

É uma maneira de reduzir a complexidade de vários if e else encadeados, caso a condição que está sendo testada for verdadeira o bloco de código é executado. O comando break termina a execução do switch.

Códigos-fonte do roteiro

Atividade 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
```

```

{
    setlocale(LC_ALL, "Portuguese");

    // Variáveis que vão ser utilizadas durante a execução do programa
    float numero;
    float soma;
    float media;
    int quantidade = 0;

    // Vai continuar executando até que o usuário digite um número diferente de zero
    while(numero != 0){
        printf("Digite um número: ");
        scanf("%f", &numero);
        if(numero > 0){
            soma += numero;
            quantidade++;
        }
    }

    media = soma / quantidade;
    // Média formatada em duas casas decimais
    printf("Media: %.2f\n", media);

    return 0;
}

```

Atividade 2:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Quantidade de linhas que "árvore" vai ter
    int linhas;

```

```
printf("Quantidade de linhas: ");
scanf("%d", &linhas);
// Árvore feita com um laço aninhado
int l, k;
for (l = 1; l <= linhas; l++){

    for (k = 1; k <= l; k++){
        printf("*");
    }
    printf("\n");
}

return 0;
}
```

Exemplo 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Inicialização da variável
    int i = 0;
    // Loop while
    while(i <= 5){
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```


Exemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Variável inicial
    int valor = 0;
    while(valor != -1){
        printf("Digite um número (-1 para sair): ");
        scanf("%d", &valor);
    }
    printf("Programa encerrado.\n");

    return 0;
}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Inicialização da variável
    char opcao;
    // do while, vai executar pelo menos uma vez
    do {
        printf("Deseja continuar? (s/n): ");
        scanf(" %c", &opcao);
    } while (opcao == 's' || opcao == 'S');

    printf("Fim do programa.\n");

    return 0;
}
```

```
}
```

Exemplo 4:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    // Variável inicial
    int l;
    // Loop de 1 até 5
    for (l = 1; l <= 5; l++){
        printf("Valor: %d\n", l);
    }

    return 0;
}
```

Exemplo 5:

```
int o, j;

for(o = 1; o <= 10; o++){
    printf("\nTabuada do %d:\n", o);

    for(j = 1; j <= 10; j++){
        printf("%d x %d = %d\n", o, j, o * j);
    }
}
```

Referências

<https://linguagemc.com.br/o-comando-while-em-c/>

<https://linguagemc.com.br/comando-do-while/>

<https://linguagemc.com.br/a-estrutura-de-repeticao-for-em-c/>

<https://linguagemc.com.br/o-comando-continue/>

<https://linguagemc.com.br/o-comando-switch-case-em-c/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Vetores (Arrays Unidimensionais)	Relatório 5
--	--	--------------------

Vetores (arrays unidimensionais)

Vetores é uma estrutura de dados que armazena uma quantidade de dados do mesmo tipo, para localizar a posição que um item está dentro de um vetor, usamos o índice, que é um número inteiro que guarda a posição dos itens. Utilizando eles conseguimos manipular uma grande quantidade de dados com apenas uma variável.

Operações com vetores

Exemplo de declaração de um vetor com 10 itens do tipo float: float vet[10], podemos também declarar e inicializar um vetor dessa forma: int numeros[5] = {1, 2, 3, 4, 5}. Nos códigos fonte abaixo, você verá diversos tipos de exemplos utilizando vetores.

Códigos-fonte do roteiro

Demonstração prática e exemplo 1 :

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Demonstração prática
    int not[5];
    not[0] = 1;
    not[1] = 2;
    not[2] = 3;
    not[3] = 4;
    not[4] = 5;
    // exemplo 1
    int idades[3] = {10, 20, 30};
    return 0;
}
```

Exemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    int notas[2];
    notas[0] = 10;
    notas[1] = 30;

    return 0;
}
```

Exemplo 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // exerc 3
    int tam;

    printf("Digite o tamanho do vetor: ");
    scanf("%d", &tam);

    int vetor_exemplo[tam];

    return 0;
}
```

Exemplo 4, 5 e 6:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    //exemplo 4
    int i, vetor[10];
```

```

for(i = 0; i < 10; i++){
    printf("Digite o %d valor: ", i+1);
    scanf("%d", &vetor[i]);
}

//exemplo 5
printf("%d\n", vetor[0]);
printf("%d\n", vetor[1]);

//Exemplo 6

int tamanho = sizeof(vetor)/ sizeof(vetor[0]);

int j = 0;
for (j; j < tamanho; j++){
    printf("%d ", vetor[j] );
}
return 0;
}

```

Exemplo 7:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    int nota[5];

    nota[0] = 10;
    nota[1] = 8;
    nota[2] = 9;
    nota[3] = 7;
    nota[4] = 6;

    printf("Nota da terceira posição: %d\n", nota[2]);
    return 0;
}

```

Exemplo 8:

```

#include <stdio.h>

```

```

#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    int t, tamanho_vetor = 3;
    int array[tamanho_vetor];

    array[0] = 1;
    array[1] = 2;
    array[2] = 3;

    for (t = 0; t < tamanho_vetor; t++){
        printf("%d ", array[t]);
    }

    return 0;
}

```

Exemplo 9:

```

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");

    int s, valores[5], soma = 0;
    float media;

    for(s = 0; s < 5; s++){
        printf("\nDigite o valor %d: ", s + 1);
        scanf("%d", &valores[s]);
        soma += valores[s];
    }

    media = soma / 5.0;

    printf("\nSoma: %d\n", soma);
    printf("\nMédia: %.2f\n", media);
}

```

```
return 0;

}
```

Atividade Proposta 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
int main()
{
    setlocale(LC_ALL, "Portuguese");
    // Variáveis
    int vet[10], total = 0, v;
    float media_vet;
    // Armazenando os dados
    for (v = 0; v < 10; v++){
        printf("\nDigite um valor: ");
        scanf("%d", &vet[v]);
        total += vet[v];
    }
    // Média
    media_vet = total / 10.0;

    printf("\nMédia: %.2f", media_vet);
    printf("\nSoma: %d", total);
    printf("\nValores pares: ");

    //Números pares armazenados no vetor
    int n;
    for (n = 0; n < 10; n++){
        if(vet[n] % 2 == 0){
            printf("%d ", vet[n]);
        }
    }

    return 0;
}
```

Referências

<https://linguagemc.com.br/vetores-ou-arrays-em-linguagem-c/>

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Matrizes (Arrays Bidimensionais)	Relatório 6
--	--	--------------------

Resumo Teórico: Desenvolva uma explicação teórica, com base no conteúdo estudado, utilizando suas próprias palavras, sobre os seguintes tópicos:

- Matrizes (arrays bidimensionais).
- Manipulação de matrizes: acesso e operações.
- o relatório deve conter prints da execução, caso possível, como forma de comprovação.

Códigos-Fontes Comentados:

- Inserir o *código-fonte* completo do roteiro 6, atividade proposta 1.
- Comentar as principais linhas do código.

Conclusão: Descreva como a aula contribuiu para o seu entendimento sobre matrizes e explique a importância desses elementos na construção de programas. Cite possíveis aplicações práticas.

Referências: O aluno deverá colocar o nome dos livros e sites utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resum o Teórico	3,0	Clareza e correlação correta entre conceitos. Conclusão bem definida.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).

Funcionalidade do Código	2,0	Execução sem erros, entrada e saída corretas.
Criatividade e Aprimoramentos	2,0	Funcionalidades extras, simulações prints de tela.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Funções em C	Relatório 7
--	--	--------------------

Orientações: A atividade pode ser realizada individualmente ou em grupo, porém cada aluno deverá entregar seu próprio relatório, com 2 a 3 páginas, contendo:

Resumo Teórico: Desenvolva uma explicação teórica, com base no conteúdo estudado, utilizando suas próprias palavras, sobre os seguintes tópicos:

- Funções em C.
- Chamada de funções e passagem de parâmetros.
- Escopo de variáveis.

Códigos-Fontes Comentados:

- Inserir o *código-fonte* completo do roteiro 7, atividade proposta 1.
- Comentar as principais linhas do código.
- Aplicar as melhorias propostas (opcional, mas recomendável).

Conclusão: Descreva como a aula contribuiu para o seu entendimento sobre operadores e explique a importância desses elementos na construção de programas.

Referências: O aluno deverá colocar o nome dos livros e sites utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resum o Teórico	3,0	Clareza e correlação correta entre conceitos. Conclusão bem definida.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).

Funcionalidade do Código	2,0	Execução sem erros, entrada e saída corretas.
Criatividade e Aprimoramentos	2,0	Funcionalidades extras, simulações prints de tela.

Instituto de Ciências Exatas e Tecnologia	Disciplina: Programação Estruturada em C Título da Aula: Ponteiros e Manipulação de Arquivos	Relatório 8
--	---	--------------------

Orientações: A atividade pode ser realizada individualmente ou em grupo, porém cada aluno deverá entregar seu próprio relatório, com 2 a 3 páginas, contendo:

Resumo Teórico: Desenvolva uma explicação teórica, com base no conteúdo estudado, utilizando suas próprias palavras, sobre os seguintes tópicos:

- Definição ponteiros em C.
- Manipulação de arquivos (leitura e escrita em arquivos).

Códigos-Fontes Comentados:

- Inserir o *código-fonte* completo do roteiro 8, atividade proposta 1.
- Comentar as principais linhas, ressaltando o uso de operadores.

Conclusão: Descreva como a aula contribuiu para o seu entendimento sobre ponteiros e arquivos e explique a sua importância na construção de programas.

Referências: O aluno deverá colocar o nome dos livros e sites utilizados para a realização da atividade. As regras para fazer referência ao material utilizado deverão ser de acordo com a ABNT.

CRITÉRIOS PARA AVALIAÇÃO

Critério	Peso	Descrição
Clareza do Resum o Teórico	3,0	Clareza e correlação correta entre conceitos. Conclusão bem definida.
Organização e Comentários do Código	3,0	O código deve estar indentado corretamente, usar nomes de variáveis adequados e conter comentários informativos (quando necessários).

Funcionalidade do Código	2,0	Execução sem erros, entrada e saída corretas.
Criatividade e Aprimoramentos	2,0	Funcionalidades extras, simulações e prints de tela.